SWIFT LOGISTICS

— MIDDLEWARE ARCHITECTURE REPORT

DOCUMENT TYPE

Technical Architecture Report

MODULE CODE

SCS 2314

MODULE NAME

Middleware Architecture

PROJECT NAME

Capstone

ARCHITECTURE

Microservices

MESSAGING

Event-Driven

GROUP MEMBERS

23001631
23002085
23000139
23001208
23001641
23002052

01 INTRODUCTION TO THE SOLUTION

1.1 BACKGROUND

Swift Logistics (Pvt) Ltd. is a rapidly growing last-mile delivery company operating in Sri Lanka. The organization relies on three independent systems:

- Client Management System (CMS) – SOAP/XML-based legacy system
- Route Optimization System (ROS) – REST-based third-party cloud API
- Warehouse Management System (WMS) – Proprietary TCP/IP real-time system

These systems operate in isolation, leading to:

- Data silos
- Manual coordination
- Inconsistent order status updates
- Poor scalability during high-volume events

1.2 OBJECTIVE OF THE PROPOSED SOLUTION

The objective of this project is to design and implement a scalable, resilient, and secure middleware architecture that enables seamless integration between heterogeneous enterprise systems. The proposed solution aims to bridge multiple legacy and modern services using a distributed microservices-based approach.

The system is specifically designed to:

- Integrate heterogeneous systems using different communication protocols such as SOAP (XML), TCP/IP, and REST (JSON).
- Enable real-time tracking of orders and deliveries across distributed components.
- Support high-volume asynchronous processing through event-driven communication using a message broker.
- Ensure transaction consistency distributed across independent services while maintaining service autonomy.
- Utilize only open-source technologies to ensure cost-effectiveness, flexibility, and extensibility.

GOAL Demonstrate how middleware can modernize legacy enterprise environments without requiring major system rewrites.

02 PROPOSED TECHNOLOGY STACK AND JUSTIFICATION

LAYER	TECHNOLOGY	JUSTIFICATION
Backend Framework	FastAPI	High-performance async Python framework
ASGI Server	Uvicorn	Lightweight production-ready server
Database	PostgreSQL	ACID compliance, reliability
Messaging	RabbitMQ	Reliable message broker for async workflows
SOAP Integration	FastAPI	SOAP/XML support for CMS
HTTP Client	HTTPX	Async REST integration with ROS
Real-Time	WebSockets	Live tracking updates
Containerization	Docker + Compose	Easy orchestration
Service Discovery	Custom Registry	Lightweight dynamic registration

2.1 WHY PYTHON AND FASTAPI

We selected **Python** as the primary programming language because it is widely used in modern backend development and has strong support for integration, asynchronous processing, and rapid development. Since this project required integrating multiple systems using different protocols (SOAP, REST, TCP), Python provided flexible libraries such as `Zeep, HTTPX, and Pika` that simplified these integrations.

For the web framework, we selected **FastAPI** because it is lightweight, high-performance, and supports asynchronous programming. In a logistics system where many orders may arrive at the same time (especially during events like Black Friday or Avurudu sales), the system must handle many concurrent requests efficiently. FastAPI is built on ASGI and supports async functions, which makes it suitable for non-blocking operations such as message publishing, database operations, and external API calls.

Another important reason for selecting FastAPI is its automatic request validation using Pydantic. This ensures that incoming order data is properly validated before being processed, which improves system reliability and security.

2.2 WHY POSTGRESQL AND SQLALCHEMY

We selected **PostgreSQL** as the primary database because it is open-source, reliable, and ACID-compliant. In a logistics system, data consistency is extremely important — an order must not disappear or become partially saved if a failure occurs. PostgreSQL ensures transaction reliability and data integrity.

We used **SQLAlchemy** as the ORM (Object Relational Mapper) because it provides a clean abstraction layer between the application logic and the database. Instead of writing raw SQL queries, we define models in Python, which makes the code easier to maintain and less error prone. This also protects against SQL injection attacks.

Another reason for selecting PostgreSQL is its scalability. As Swift Logistics grows, the database can handle large volumes of records and can be scaled vertically or horizontally if needed.

2.3 WHY RABBITMQ FOR MESSAGING

One of the key requirements of the system is handling high volumes of incoming orders without blocking the client portal. For this reason, we selected RabbitMQ as our message broker.

RabbitMQ allows services to communicate asynchronously. When an order is placed, instead of waiting for the CMS, WMS, and ROS systems to respond, the Order Service publishes a message to RabbitMQ. Other services consume the message and process it independently. This makes the system much faster and more responsive.

We chose RabbitMQ instead of Kafka because RabbitMQ is easier to configure and manage for medium-scale systems. Kafka is powerful but more complex and better suited for extremely large-scale event streaming systems. Since Swift Logistics is growing but not yet operating on a global enterprise scale, RabbitMQ is a practical and efficient choice.

RabbitMQ also supports durable queues, acknowledgments, and retry mechanisms. This ensures that messages are not lost even if a service crashes temporarily.

2.4 WHY FASTAPI FOR SOAP INTEGRATION

The CMS system used in this project exposes a SOAP-based XML API, which is considered a legacy communication protocol. Since the rest of our system is designed using REST and JSON, we needed a way to integrate SOAP-based CMS without changing its implementation. For this purpose, we used FastAPI to build an adapter service.

FastAPI does not natively support SOAP, but it provides a flexible and high-performance framework that allows us to handle HTTP requests and responses efficiently. Within the FastAPI

adapter, we manually construct the required SOAP XML messages and send them to the CMS system. Once the CMS returns a SOAP response, the adapter parses the XML and converts it into JSON before returning it to other services.

This approach allows the rest of the system to remain fully REST-based, while SOAP communication is handled internally within the adapter layer. As a result, the legacy CMS system remains unchanged, and the protocol differences are hidden from other microservices.

Using FastAPI also provides additional benefits such as automatic API documentation through Swagger UI, structured request validation, and asynchronous request handling. Overall, FastAPI enabled us to integrate a legacy SOAP system into a modern microservices architecture while maintaining clean separation of concerns and keeping the overall system design consistent.

2.5 WHY HTTPX FOR REST INTEGRATION

The Route Optimization System (ROS) provides a REST API. To communicate with it asynchronously, we used **HTTPX**, which supports Async HTTP calls. This allows the system to call the ROS API without blocking other operations. Since route optimization can take time, asynchronous HTTP calls help maintain high performance during peak traffic.

2.6 WHY WEBSOCKETS FOR REAL-TIME UPDATES

One of the key functional requirements is real-time tracking. When a driver marks a package as delivered, the client portal must immediately show this update.

For this purpose, we implemented **WebSockets**. Unlike traditional HTTP requests, WebSockets maintain a persistent connection between the server and the client. This allows the server to push updates instantly without waiting for the client to refresh the page. This improves user experience and makes the system feel modern and responsive.

2.7 WHY DOCKER AND DOCKER COMPOSE

Since the system consists of multiple independent services, manually running each service would be complex and error prone. Therefore, we used Docker to containerize each microservice and Docker Compose to orchestrate them.

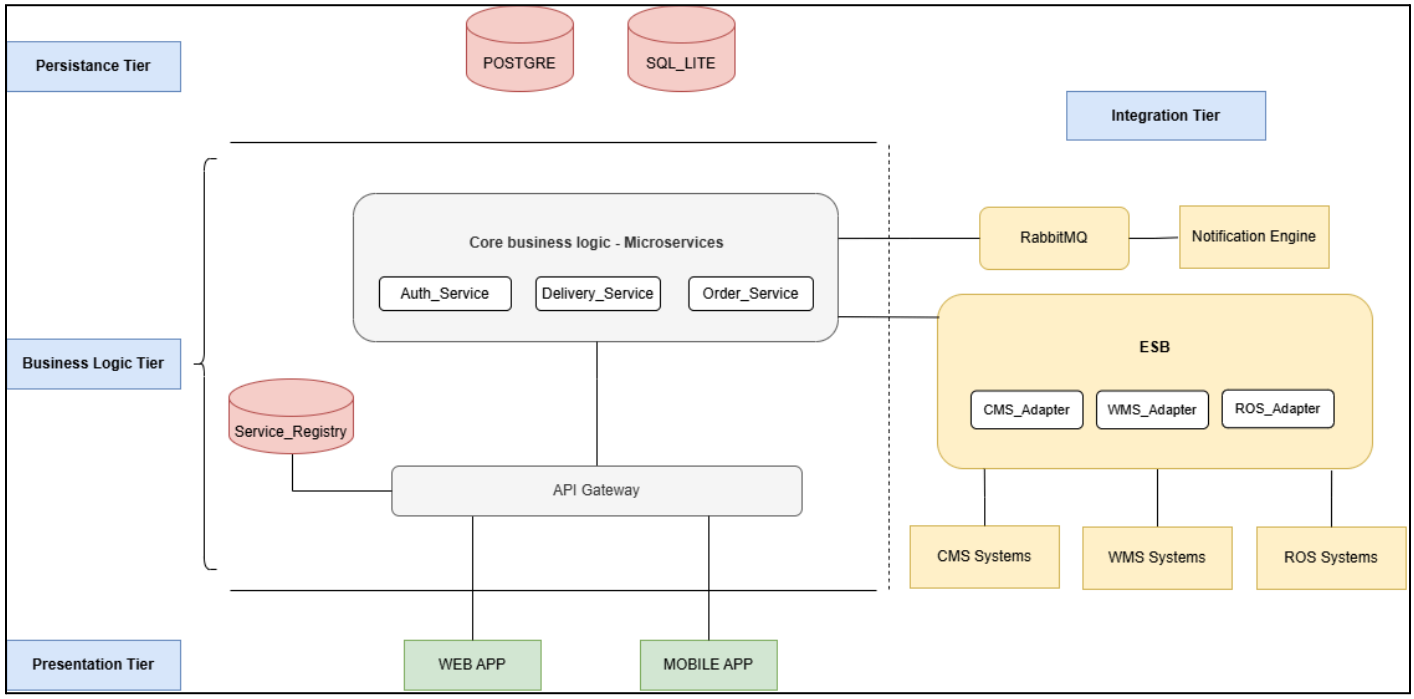
Docker ensures that:

- Each service runs in an isolated environment
- Dependence is consistent across machines
- Deployment is reproducible

Docker Compose allows us to define all services (database, message broker, adapters, etc.) in a single YAML file. This simplifies deployment and makes the system easier to manage. This approach also supports scalability — in the future, individual services can be scaled by increasing container instances.

03 ARCHITECTURE OF THE SOLUTION

3.1 CONCEPTUAL ARCHITECTURE



3.2 ARCHITECTURAL CHARACTERISTICS

The proposed architecture is designed based on modern distributed system principles. The following key characteristics define the system:

3.2.1 Loosely Coupled Services

Each microservice operates independently and communicates through well-defined interfaces or message queues. This reduces interdependence and allows services to evolve without impacting others.

3.2.2 Event-Driven Communication

RabbitMQ is used as a message broker to enable asynchronous communication between services. Instead of direct service-to-service calls, events such as OrderCreated or OrderShipped are published and consumed by interested services. This improves scalability and resilience.

3.2.3 Database per Service Pattern

Each microservice maintains its own dedicated database. This prevents tight data coupling and ensures that services remain autonomous in managing their own data.

3.2.4 Adapter-Based Legacy Integration

Legacy systems such as CMS (SOAP-based) and WMS (TCP-based) are integrated using adapter services. These adapters translate modern REST/JSON communication into legacy-compatible formats without modifying the original systems.

3.2.5 Containerized Deployment

All services are containerized using Docker and orchestrated using Docker Compose. This ensures portability, simplified deployment, and environment consistency across development and testing platforms.

3.3 IMPLEMENTATION ARCHITECTURE

The implementation follows microservices-based architecture deployed in a containerized environment.

3.3.1 Core Microservices

- Service Registry (Port 8001) – Responsible for service discovery and registration. Enables dynamic lookup of service endpoints.
- Order Service (Port 8004) – Acts as a central business service responsible for handling order placement, status updates, and publishing domain events.
- Delivery Service (Port 8005) – Manages delivery assignment, driver notifications, and delivery tracking.
- CMS Adapter (Port 8003) – Integrates with the legacy CMS system using SOAP/XML. Handles login validation and order persistence within the CMS.
- WMS Adapter (Port 8002) – Connects to the legacy Warehouse Management System using TCP/IP protocol. Responsible for transmitting warehouse-related order information.

3.3.2 Infrastructure Components

- RabbitMQ – Serves as the asynchronous messaging backbone for inter-service communication.
- PostgreSQL – Used as the relational database system for microservices requiring persistent storage.
- Docker Compose – Manages container orchestration and ensures consistent multi-container deployment.

This layered structure separates business logic, integration logic, and infrastructure concerns effectively.

04 ALTERNATIVE ARCHITECTURES CONSIDERED

4.1 ALTERNATIVE 1 — CENTRALIZED ESB ARCHITECTURE

Technologies Considered

- Apache ServiceMix
- Mule ESB

Architectural Model

This alternative was based on a centralized Enterprise Service Bus (ESB) model, where:

- A central integration engine manages all routing and transformation logic.
- Message orchestration is handled centrally.
- Data transformation pipelines are managed within the ESB.

ADVANTAGES	DISADVANTAGES
Strong governance and centralized control.	Central orchestration reduces service autonomy.
Unified monitoring and logging capabilities.	Integration logic becomes tightly coupled within the ESB.
Built-in transformation pipelines for protocol conversion.	Risk of performance bottlenecks due to centralized processing.
	Increased long-term maintenance complexity.

WHY REJECTED Risk of creating a central bottleneck and reducing microservice independence. The decentralized approach with RabbitMQ better serves the loosely coupled, event-driven objectives of this project.

4.2 ALTERNATIVE 2 — FULLY EVENT-DRIVEN KAFKA-BASED ARCHITECTURE

Technology Considered

- Apache Kafka

Architectural Model

This alternative design was based on a fully event-driven streaming architecture using Apache Kafka as the central messaging backbone.

The architectural approach included event sourcing, where all system state changes are stored as immutable events, a choreography-based Saga pattern for distributed transaction management, and high-throughput distributed event streaming for large-scale data processing. In this model, services would react to published events independently without centralized orchestration.

ADVANTAGES	DISADVANTAGES
High scalability is suitable for large enterprise workloads.	Significant operational overhead compared to RabbitMQ.
Durable event log with long-term storage capabilities.	Complex cluster configuration and maintenance.
Strong replay capability for system state reconstruction.	Higher debugging difficulty due to distributed log partitions.
Excellent support for distributed streaming pipelines.	Throughput capacity exceeds projected load requirements.

WHY REJECTED

Projected system scales do not justify the additional operational complexity. RabbitMQ provides sufficient async messaging while maintaining lower configuration and maintenance overhead.

05 ARCHITECTURAL & INTEGRATION PATTERNS USED

5.1 MICROSERVICES PATTERN

Each service owns its logic, has clear boundaries, and can scale independently.

We adopted microservices architecture because Swift Logistics requires scalability, flexibility, and resilience. In a monolithic system, all functionalities are tightly connected — if one part fails, the entire system may become unavailable.

In contrast, microservices divide the system into smaller, independent services such as Order Service, Delivery Service, CMS Adapter, and WMS Adapter. Each service handles a specific responsibility.

This provides several advantages:

- Independent deployment
- Independent scaling
- Fault isolation
- Better maintainability

For example, during high traffic events, we can scale only the Order Service without affecting other services.

5.2 ADAPTER PATTERN

Used for CMS (SOAP → REST conversion), WMS (TCP → HTTP/Events), and ROS (REST proxy adapter).

The system integrates three heterogeneous systems: CMS (SOAP/XML), ROS (REST/JSON), and WMS (Proprietary protocol). To handle these differences, we used the Adapter Pattern. Each legacy system has a dedicated adapter service that translates between our internal format and the external protocol.

This ensures that core services remain clean and protocol independent. Legacy systems can be replaced in the future without affecting the entire system, and changes in one external system do not break the overall architecture. This significantly improves system maintainability.

5.3 SERVICE REGISTRY PATTERN

Centralized discovery mechanism. In a containerized microservices environment, service IP addresses may change dynamically. Hardcoding service addresses would make the system fragile.

Therefore, we implemented a Service Registry. When a service starts, it registers with the registry. Other services can query the registry to discover available services.

This provides: Dynamic service discovery, improved scalability, and reduced configuration complexity.

5.4 PUBLISH-SUBSCRIBE PATTERN

Implemented using RabbitMQ and used for: order created events, delivery status updates, and route update notifications.

We implemented the Publish–Subscribe pattern using RabbitMQ. When an event such as "Order Created" occurs, the Order Service publishes a message. Other services subscribe and react accordingly.

This decouples services from each other. The Order Service does not need to know which services consume the message. This improves scalability, flexibility, loose coupling, and system responsiveness.

5.5 SAGA PATTERN (DISTRIBUTED TRANSACTIONS)

Order creation triggers the following sequence:

1. CMS update
2. WMS registration
3. ROS route optimization

If any step fails, compensation logic is executed and the order status will be rolled back. Since traditional database transactions cannot span multiple services, we used the Saga Pattern.

In this approach, each step completes independently. If one step fails, compensating actions are triggered to undo previous steps.

EXAMPLE If CMS succeeds but ROS fails, the order is marked as "Pending Route Assignment" and the retry mechanism is triggered. This ensures eventual consistency without using complex distributed locks.

5.6 CIRCUIT BREAKER PATTERN (CONCEPTUALLY DESIGNED)

If ROS becomes unavailable:

- Order Service does not block
- Request is retried
- Fallback logic is used

This prevents cascading failures.

06 PROTOTYPE IMPLEMENTATION

6.1 IMPLEMENTED COMPONENTS

The prototype implementation includes the following components:

- Order submission REST endpoint
- CMS SOAP integration implemented via FastAPI
- ROS REST integration using HTTPX client
- WMS adapter simulation for TCP-based communication
- RabbitMQ-based asynchronous event publishing and consumption
- PostgreSQL database persistence for core services
- Real-time WebSocket notification mechanism
- Docker-based container orchestration using Docker Compose

Each component was implemented independently and deployed as a containerized microservice.

6.2 ORDER FLOW

The implemented order processing workflow operates as follows:

4. The client submits an order through the REST API endpoint exposed by the Order Service.
5. The Order Service validates and saves the order in PostgreSQL.
6. An event is published to RabbitMQ indicating that a new order has been placed.
7. The CMS Adapter processes the order via a SOAP-based integration call.
8. The ROS Adapter optimizes the delivery route using REST-based communication.
9. The WMS Adapter registers the package in the warehouse system.
10. The Delivery Service updates the delivery status.
11. A WebSocket notification is pushed to the client interface to provide real-time updates.

This flow demonstrates hybrid integration across SOAP, REST, TCP, and message-driven components within a unified middleware architecture.

6.3 REAL-TIME TRACKING

Real-time tracking functionality is implemented using WebSockets. When a delivery status change occurs, an event is published to RabbitMQ. The Delivery Service consumes the event. The WebSocket server pushes the updated status to the connected client interface.

WebSockets provide persistent bi-directional communication, allowing live tracking updates without repeated HTTP polling. This improves responsiveness and reduces unnecessary network traffic.

6.4 TRANSACTION MANAGEMENT STRATEGY

The system follows a Saga-based eventual consistency model for distributed transaction management. Instead of relying on two-phase commit protocols, each service executes its local transaction and publishes an event upon completion. If a failure occurs, compensating actions are triggered.

6.5 RECOVERY STRATEGY

Scenario 1: CMS Succeeds but ROS Fails

A compensation event is triggered. Order status is updated to "Pending Route Assignment". A retry mechanism is scheduled using Celery background tasks.

Scenario 2: WMS Processing Fails

The message is redirected to a Dead Letter Queue (DLQ). Retry logic is executed asynchronously.

6.6 MESSAGE DURABILITY

RabbitMQ persistent queues are configured. Message acknowledgement mechanisms ensure reliable delivery. Unacknowledged messages are re-queued automatically.

6.7 IDEMPOTENCY

Order IDs are used as unique identifiers to prevent duplicate processing. If the same message is received multiple times, the service validates whether the order has already been processed before executing the operation.

07 SCALABILITY & RESILIENCE

7.1 SCALABILITY MECHANISMS

The system supports scalability through:

- Horizontal scaling of containerized services
- Stateless microservice design
- Message queue buffering to handle traffic spikes
- Database indexing for optimized query performance

Each service can be independently scaled without impacting others.

7.2 RESILIENCE FEATURES

To ensure system reliability, the following resilience mechanisms are implemented:

- Health checks configured within Docker Compose
- RabbitMQ durable queues to prevent message loss
- Service Registry failover handling
- Asynchronous retry mechanisms using Celery
- Fault isolation between microservices to prevent cascading failures

These mechanisms collectively enhance system stability and fault tolerance.

08 SECURITY CONSIDERATIONS

8.1 COMMUNICATION SECURITY

- HTTPS-ready configuration
- TLS for service-to-service communication
- Secure RabbitMQ credentials

8.2 AUTHENTICATION & AUTHORIZATION

- JWT-based authentication (architecturally designed)
- Role-Based Access Control (Client, Driver, Admin)

8.3 DATA PROTECTION

- PostgreSQL password protection
- Environment variable secrets
- Input validation via Pydantic
- SQL Injection prevention via ORM

8.4 API PROTECTION

- Rate limiting (conceptually designed)
- API Gateway validation
- Request schema validation

Security is a critical requirement because the system handles client data, delivery information, and internal operations. All communication is designed to run over HTTPS with TLS encryption. This prevents attackers from intercepting sensitive information.

Authentication is designed using JWT tokens. This ensures that only authorized users (clients, drivers, admins) can access the system. Input validation is handled by Pydantic models, preventing malformed data from entering the system.

Database access is secured using credentials stored in environment variables rather than hardcoded values. RabbitMQ is configured with authentication credentials to prevent unauthorized message publishing or consumption.

SECURITY GOAL These measures ensure confidentiality, integrity, and availability of the system.

09 DEPLOYMENT STRATEGY

9.1 OVERVIEW

The system is deployed using **Docker Compose** to orchestrate a multi-container microservices architecture. Each functional component runs inside an isolated Docker container, enabling reproducibility, portability, and consistent execution across development and staging environments.

The deployment environment consists of:

- Application services
- Legacy system containers
- Message broker
- Relational database
- Adapter layer for legacy integration

Docker Compose manages container lifecycle operations, service dependency ordering, internal networking, and environment configuration. This approach ensures that the complete system can be deployed using a single command while maintaining modular separation between services.

9.2 ARCHITECTURE OVERVIEW

The deployment follows a containerized service-oriented architecture. Each service operates in an isolated Docker container and communicates through a shared internal Docker network.

All services are logically separated but connected via defined network interfaces, enabling structured and controlled communication between components.

10 INFRASTRUCTURE COMPONENTS

10.1 MESSAGE BROKER

The system uses **RabbitMQ** as the asynchronous communication backbone.

Purpose is to decouple microservices by eliminating direct synchronous dependencies, enable event-driven messaging patterns, and improve fault tolerance through message buffering and durability. RabbitMQ allows services to publish and consume domain events without requiring knowledge of each other's internal implementation.

10.2 DATABASE LAYER

The system uses **PostgreSQL (version 16 Alpine image)** as the relational database engine.

Key deployment features include a health check mechanism to ensure database readiness before dependent services start, and persistent storage volumes to guarantee data durability across container restarts. This configuration ensures reliable data persistence and prevents race conditions during system startup.

11 APPLICATION SERVICES

The core application services are deployed as independent containers, each responsible for a specific business capability. These services expose REST-based APIs and communicate asynchronously using RabbitMQ.

12 LEGACY SYSTEMS

The deployment includes simulated legacy enterprise systems:

- **wms_legacy**
- **ros_legacy**
- **cms_legacy**

These services expose HTTP-based APIs and represent pre-existing enterprise systems integrated through adapter services. They are intentionally isolated to demonstrate real-world legacy system integration without modifying the original systems.

13 ADAPTER LAYER

The adapter layer acts as a bridge between modern microservices and legacy systems.

wms_adapter

The WMS Adapter implements a translation layer between the legacy WMS system and the modern event-driven architecture. It communicates with wms_legacy using HTTP and RabbitMQ using AMQP protocol.

NOTE Docker Compose automatically resolves container names as hostnames within the internal network. Services can communicate using wms_legacy:port without manual IP configuration. This simplifies service discovery while maintaining network isolation.

14 NETWORKING MODEL

All services are connected to a default Docker bridge network created automatically by Docker Compose.

Characteristics:

- Containers communicate using service names as hostnames
- No external service registry is required within the container network
- Internal ports are accessible only within the Docker network
- Selected services expose mapped host ports for external client access

This networking model simplifies internal communication while preserving external security boundaries.

15 SERVICE DEPENDENCY MANAGEMENT

Docker Compose utilizes the depends_on directive with health check conditions to enforce proper startup sequencing.

Benefits:

- Prevents race conditions during container initialization
- Ensures infrastructure components (e.g., database, message broker) are ready before application services attempt connections

This improves overall deployment stability and reliability.

16 ENVIRONMENT CONFIGURATION

Service configuration is managed through environment variables defined in the Docker Compose file. Typical configuration values include:

- Database connection strings
- RabbitMQ host reference
- Legacy system hostnames and ports
- Service-specific runtime parameters

This approach allows environment-specific configuration without modifying application source code, supporting flexible deployment across different environments.

17 SCALING STRATEGY

Horizontal scaling is supported through Docker Compose using the following command:

```
docker compose up -d --scale delivery_service=3
```

Docker will launch multiple container instances of the specified service, attach them to the same internal network. Because the services are designed to be stateless, multiple instances can run concurrently without conflict.

Production Consideration

For production-grade deployments, a reverse proxy or API Gateway should be introduced to provide load balancing across service replicas, centralized authentication and authorization, SSL termination, and rate limiting and traffic management. This enhancement would further improve scalability and operational robustness.

18 CHALLENGES FACED

18.1 SOAP INTEGRATION COMPLEXITY

Problem

Integrating legacy systems that expose SOAP-based APIs introduced significant complexity compared to REST-based communication. SOAP requires strict XML schema adherence, WSDL contract interpretation, namespace management, and complex request/response structures. Minor schema mismatches resulted in request failures.

What Makes It Difficult

Unlike REST (simple JSON over HTTP), SOAP requires XML envelope wrapping, has rigid message formats, produces verbose payloads, and is harder to debug manually. Additionally, error messages were often unclear, making debugging slow.

18.2 HANDLING ASYNCHRONOUS EVENT DEBUGGING

Problem

Services communicate through RabbitMQ, which introduces asynchronous behavior. Unlike synchronous HTTP calls, message producers do not wait for consumers, failures do not immediately propagate, and message ordering may vary. This made debugging difficult.

Why It Was Difficult

In async systems, execution flow is non-linear, bugs may occur after a delay, and logs are distributed across multiple containers. Tracing an event required correlating logs across producer service, broker, and consumer service. This makes root-cause analysis of errors harder.

18.3 MANAGING DISTRIBUTED TRANSACTIONS

Problem

Some operations required coordination across the database (PostgreSQL), legacy services, and the message broker. There is no global transaction manager across these components.

Difficulties

Traditional ACID transactions only apply within a single database. When multiple services are involved, partial failures can leave the system in an inconsistent state. Rollbacks are not automatically propagated.

18.4 CONTAINER NETWORKING ISSUES

Problem

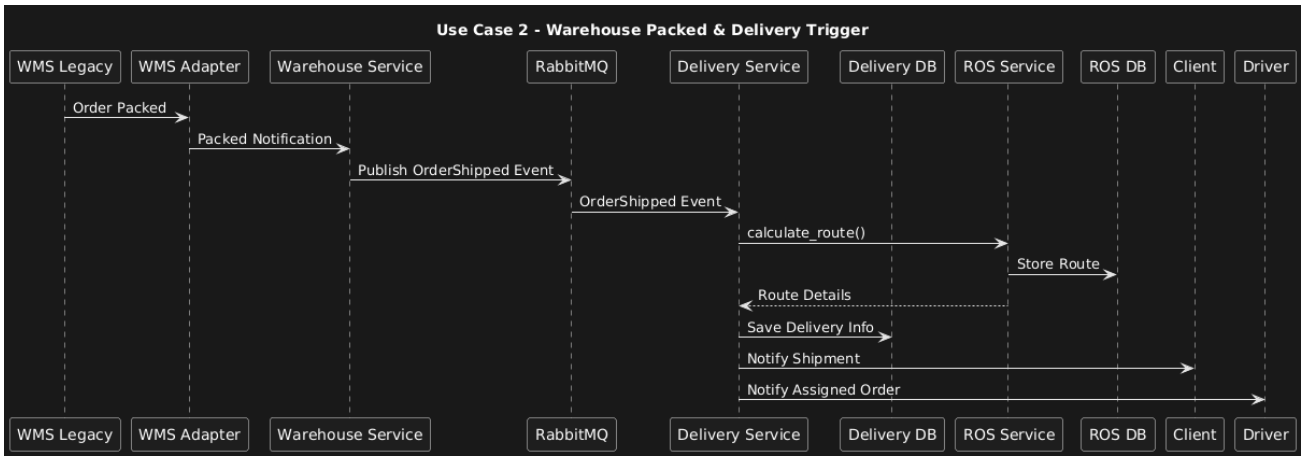
Initial configuration confusion occurred regarding localhost vs. container hostnames and host.docker.internal when doing transactions between internal and external ports.

Why It Was Difficult

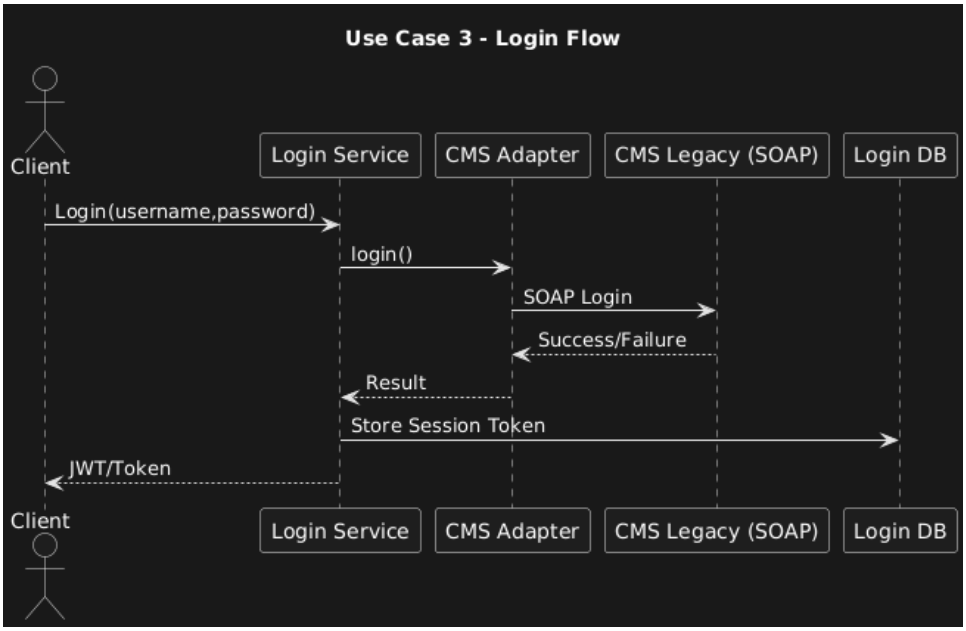
Inside Docker, local host refers to the container itself, and services must use container names (e.g., db, rabbitmq) for internal communication. External port mapping does not apply inside the Docker network.

19 DIAGRAMS

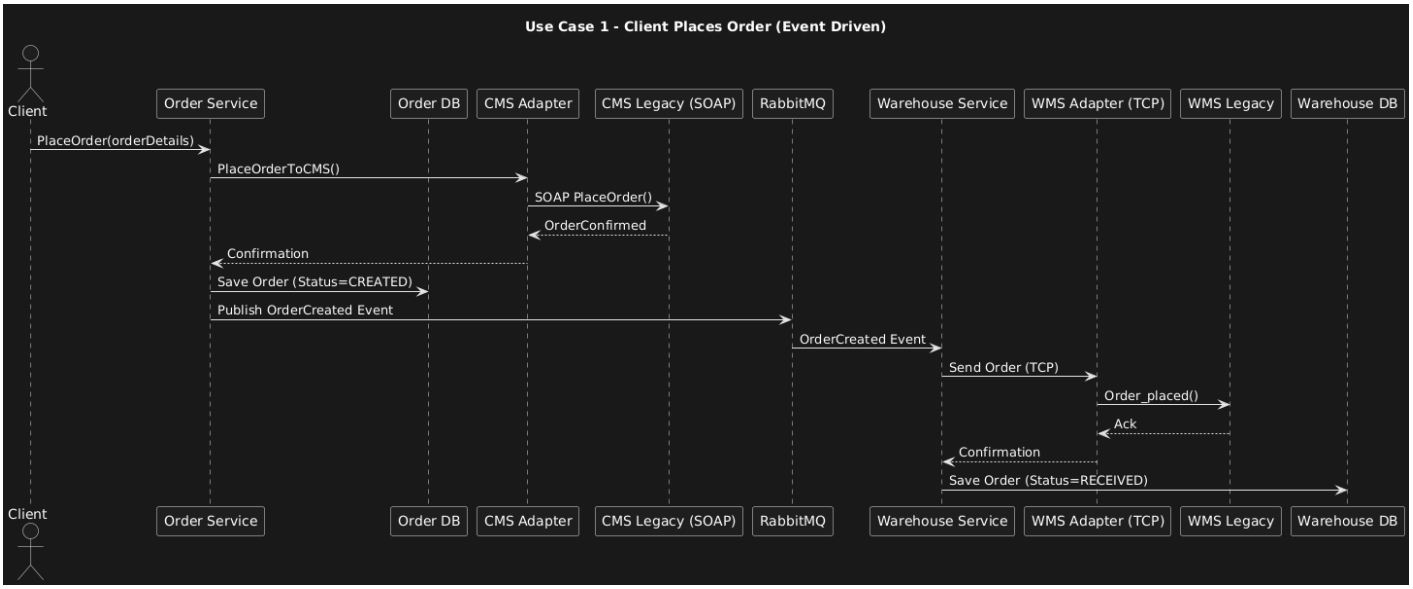
1. EVENT DIAGRAM



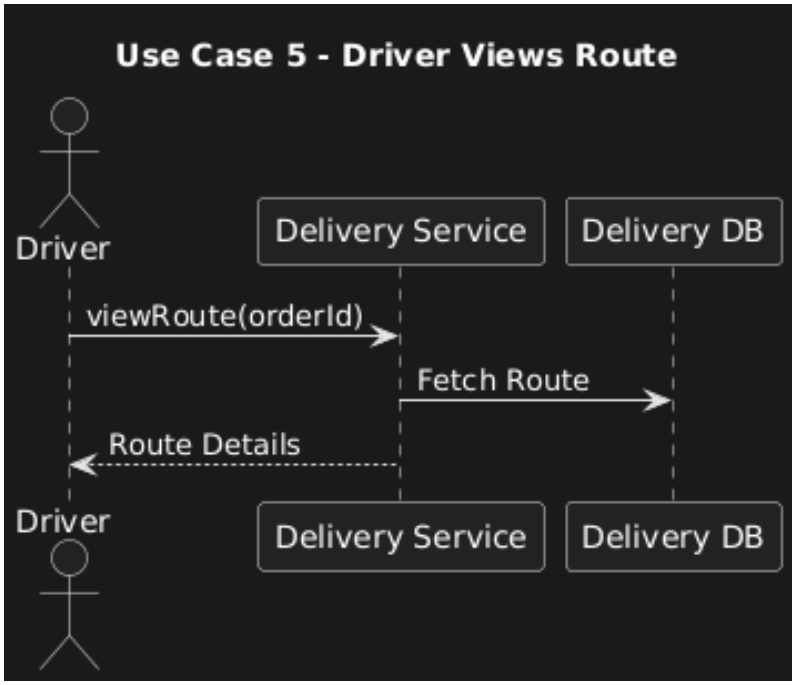
2. LOGIN FLOW USE CASE DIAGRAM



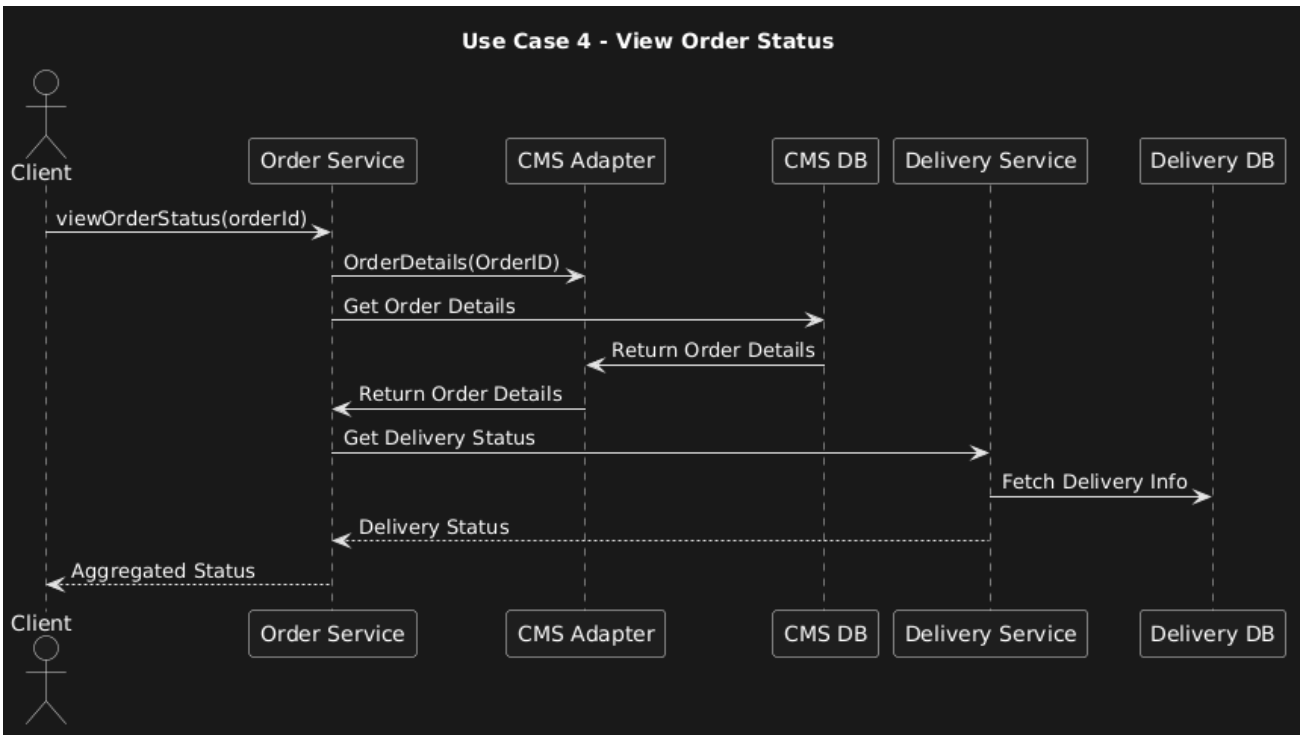
3. CLIENT PLACE ORDER (USE CASE)



4. DRIVER VIEW ROUTES (USE CASE)



5. VIEW ORDER STATUS (USE CASE)



6. SUBMIT FEEDBACK (USE CASE)